

Lab: 0101

The Day Your Circuit Learns to Remember.

From Combinational to Sequential

How a circuit gains memory — and how a clock keeps that memory sane.

CSA222: Modern Computer Architecture

What You'll Build Today

Three storage elements, in order of increasing discipline. By the end, you'll have the building block of every register on a CPU.

01

SR Latch

Two cross-coupled NORs. Holds one bit. You'll meet its forbidden state — the warning sign of all latches.

02

D Latch

Tame the SR latch with an enable. While $E=1$ the output follows D. While $E=0$ it remembers. Transparent and glitch-prone.

03

D Flip-Flop

Same idea, but it samples only on the clock's rising edge. The workhorse of every synchronous CPU.

Today is where Verilog stops being just combinational equations and starts looking like state.

Recap · Where We Are

Labs 1–4 built circuits whose outputs are pure functions of their inputs. Today the output depends on history too.

01**Combinational so far**

Adders, MUXes, decoders — outputs change the instant inputs change. No state. No clock.

02**But CPUs have state**

Registers hold partial sums between cycles. The PC remembers what to fetch next. Memory remembers everything.

03**Feedback is the answer**

Loop an output back into an input through a gate. With the right topology, the circuit holds onto a bit.

04**Then add discipline**

Pure feedback is messy — async, glitch-sensitive. The clock turns chaos into a sequence of well-defined moments.

Every circuit from today onward will have memory. Get used to thinking in time, not just in logic.

Why We Need Memory

Pure logic forgets the instant inputs change. To compute anything iterative, the circuit has to hold a value between steps.

$f(x)$

what combinational
gates compute

0 ns

how long they
remember

1 bit

the minimum unit
of state we need

∞

complexity we can
build once we have it

THE INSIGHT

Try writing a counter with only AND/OR/XOR. You can't. There's no way to ask 'what was Q a moment ago?' without something that remembers. That something is built from feedback.

Combinational vs Sequential

The single most important distinction in digital design.

COMBINATIONAL

Output = $f(\text{inputs now})$

- No memory — purely reactive.
- Same inputs always give same output.
- Examples: adders, MUXes, decoders.
- The instant inputs change, output follows.

SEQUENTIAL

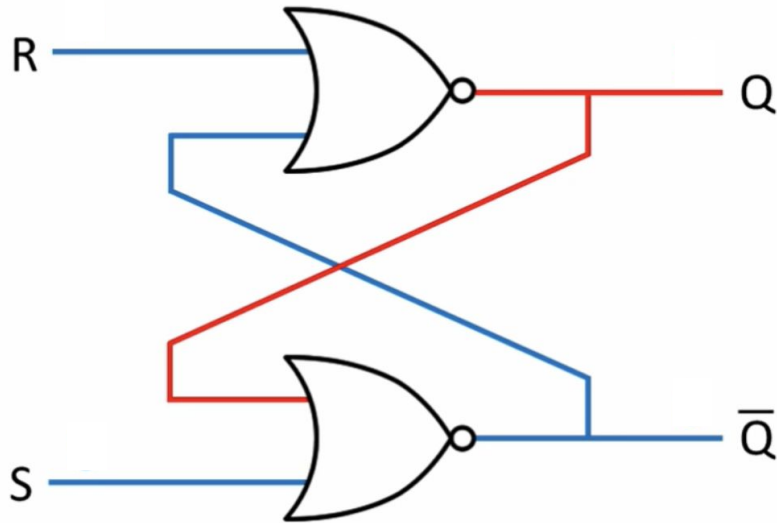
Output = $f(\text{inputs, past state})$

- Has memory — remembers history.
- Same inputs can give different outputs.
- Examples: registers, counters, FSMs.
- A clock decides WHEN state updates.

The difference is one thing: a feedback loop. Add it, and a circuit can remember.

The Idea · Feedback Creates State

Wire an output back into an input. If the topology supports two stable configurations, the circuit can be in either — and stay there.



Each gate's output feeds the OTHER gate's input — that's the cross-coupling that holds state.

WHAT YOU SEE

Two NOR gates feeding each other's inputs. Two stable configurations: $(Q=1, Q'=0)$ and $(Q=0, Q'=1)$. Either holds itself.

SR Latch · Schematic & Truth Table

Two NOR gates, four input combinations, four very different behaviours.

S	R	Q (next)	Behaviour
0	0	Q (hold)	remember
1	0	1	set
0	1	0	reset
1	1	??	forbidden

Read row 4: with S=R=1, both NORs try to drive their outputs to 0 — but then both feedbacks are 0, which would force both back to 1. The latch oscillates.

SR Latch · The Equations

Two assignments, each referring to the other's output. That mutual reference is the storage.

Q (the bit we read)

$$Q = \sim(R \mid Q')$$

Q stays 1 unless R is asserted. If R=1, Q is pulled to 0 regardless of Q'.

Q' (the inverted bit)

$$Q' = \sim(S \mid Q)$$

Q' stays 1 unless S is asserted. If S=1, Q' is pulled to 0, which lets Q rise to 1.

WHY THIS WORKS

When S=R=0 (no command), each equation simplifies to $Q = \sim Q'$, $Q' = \sim Q$. That's a consistent pair — and it's stable. Two such pairs exist; the latch sits in whichever it last entered.

SR Latch · Verilog

Two assigns, each referencing the other's wire. Verilog handles the simultaneous evaluation.

`sr_latch.v`

```
// NOR-based SR Latch
module sr_latch (
    input  wire s,
    input  wire r,
    output wire q,
    output wire qbar
);
    // Cross-coupled NORs:
    // q depends on qbar, qbar depends on q.
    assign q      = ~(r | qbar);
    assign qbar   = ~(s | q);
endmodule
```

WHY THIS COMPILES

- Each assign is continuous — it re-evaluates whenever its RHS changes.
- When you change s or r, one assign reacts, then the other, then it settles.
- With S=R=1, both keep firing forever — that's the simulator's way of telling you about the forbidden state.
- Real hardware doesn't oscillate cleanly; it lands at whichever NOR is slightly faster.

The Forbidden State · $S = R = 1$

What the truth table calls 'forbidden' isn't an error in Verilog — it's the moment the latch stops being a latch.

1

Both fire

$S=R=1$ forces both NOR outputs to 0. So $Q=0$ and $Q'=0$ — they're equal, breaking the invariant $Q \neq Q'$.

2

Release S and R

Now both inputs return to 0. The NORs feed back the previous 0's — each tries to flip to 1. They race.

3

Whoever is faster wins

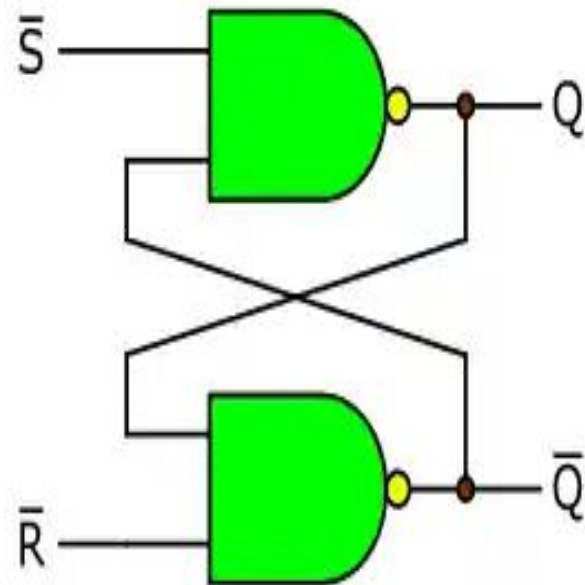
The final state depends on tiny propagation differences between the two gates. Non-deterministic — which is unusable in a CPU.

THE LESSON

Every latch design that exposes both S and R must guarantee they're never both 1. The D latch solves this by construction — R is always $\sim S$.

SR Latch, NAND Version

Same idea, built from NAND gates — but the inputs are active-LOW.



Input		Output	Action
$\bar{S}$	$\bar{R}$	Q_{NEXT}	
1	1	Q	Hold
0	1	1	Set
1	0	0	Reset
0	0	1	Illegal

NOR vs NAND LATCH

NOR latch: active-HIGH (1 = act).

NAND latch: active-LOW (0 = act).

Forbidden state flips too: for the NAND latch, $S' = R' = 0$ is the illegal one.

NAND latches are common in practice because NAND is the cheap, universal gate (recall L2).

Note the primes: S' and R' are active-LOW — a 0 triggers the action, not a 1.

SR Latch · Testbench

Walk through all four cases. The forbidden case will show oscillation in iverilog — read the log carefully.

sr_tb.v

```
module sr_tb;
  reg s, r; wire q, qbar;
  sr_latch dut(.s(s), .r(r), .q(q), .qbar(qbar));
  initial begin
    $dumpfile("sr.vcd"); $dumpvars(0, sr_tb);
    s=0; r=0; #5; // initial - unknown
    s=1; r=0; #5; // SET -> q=1
    s=0; r=0; #5; // HOLD -> q stays 1
    s=0; r=1; #5; // RESET -> q=0
    s=0; r=0; #5; // HOLD -> q stays 0
    s=1; r=1; #5; // FORBIDDEN
    s=0; r=0; #5; // metastable!
  $finish;
end
endmodule
```

WHAT TO LOOK FOR

- Steps 1–5: q follows the rules — set, hold, reset, hold.
- Step 6: q and qbar both go to 0 — illegal state.
- Step 7: after release, iverilog may keep toggling — that's the oscillation.
- If you see X (unknown) on q after step 7, the simulator gave up. That's the right behaviour.

The SR Latch Is Hard to Use

Even when you avoid the forbidden state, two things make raw SR latches dangerous in a real design.

01

Two control lines is one too many

Every consumer of this latch has to remember not to assert S and R together. That's a rule humans break.

02

Any input glitch propagates

A 1 ns spike on S during HOLD will set the latch. There's no way to ignore noise without external gating.

03

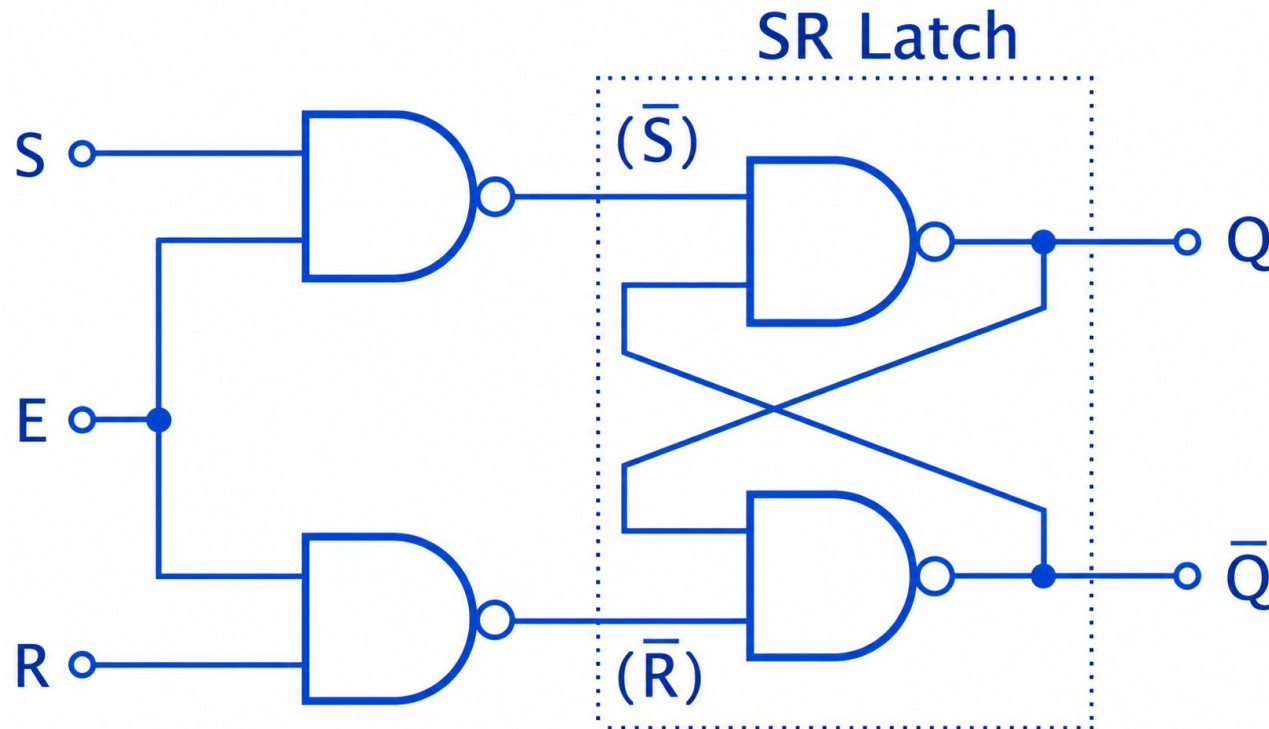
No notion of 'now'

The latch reacts to inputs the instant they change. Synchronous design needs a clock to define when things happen.

The D latch fixes problems 1 and 2. The flip-flop fixes problem 3.

Adding an Enable: The Gated Latch

Now the latch only listens when we say so — the first step toward a clock.



The **Enable (E)** controls the latch: when enabled, **S** sets $Q = 1$ and **R** resets $Q = 0$; when disabled, the previous state is retained.

WHY THIS MATTERS

When $E = 1$ (Enable): The latch responds to **S** and **R** — $S = 1$ sets $Q = 1$, while $R = 1$ resets $Q = 0$.

When $E = 0$ (Disable): The latch **holds its previous state**, so Q remains unchanged.

D Latch · One Input, One Enable

Take the SR latch. Tie $R = \sim S$ internally. Add an enable that gates everything. Now S and R can never both be 1.

$$S = D \cdot E \quad R = \sim D \cdot E$$

01

When $E = 0$

Both S and R are 0. The SR latch is in HOLD. D can wiggle all it wants — output doesn't change.

02

When $E = 1, D = 1$

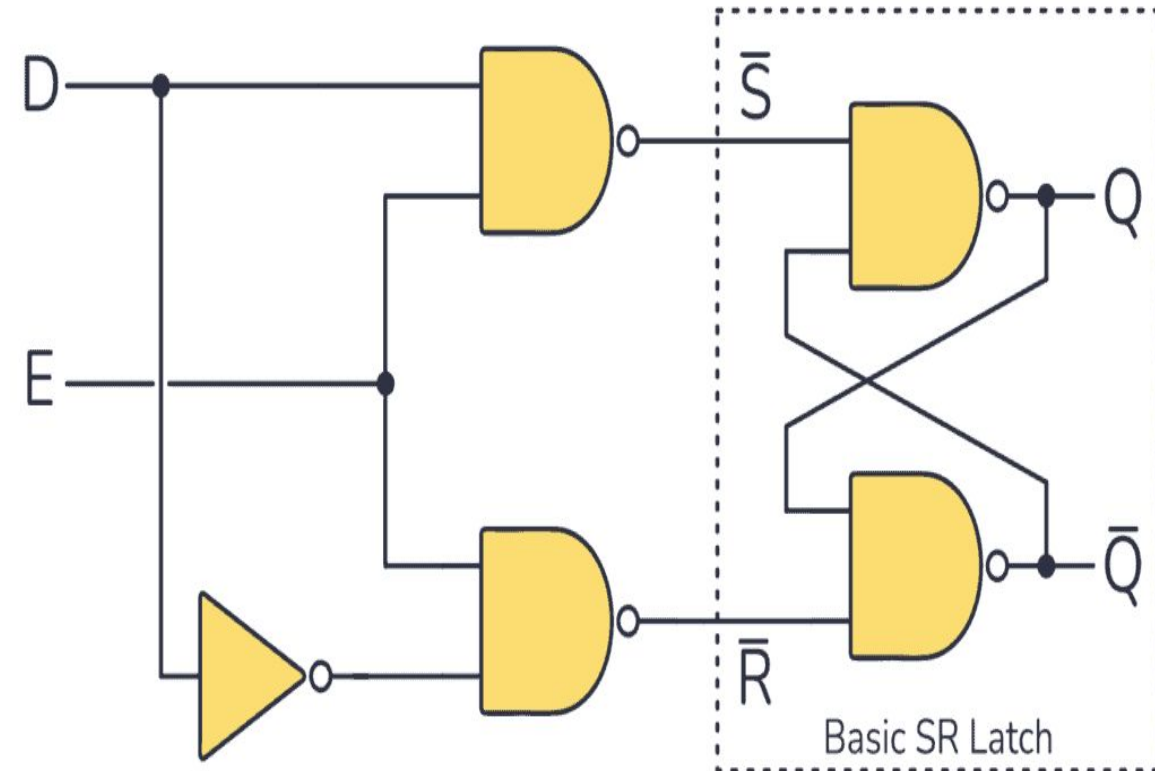
$S=1, R=0$. SR latch is SET. $Q = 1$.

03

When $E = 1, D = 0$

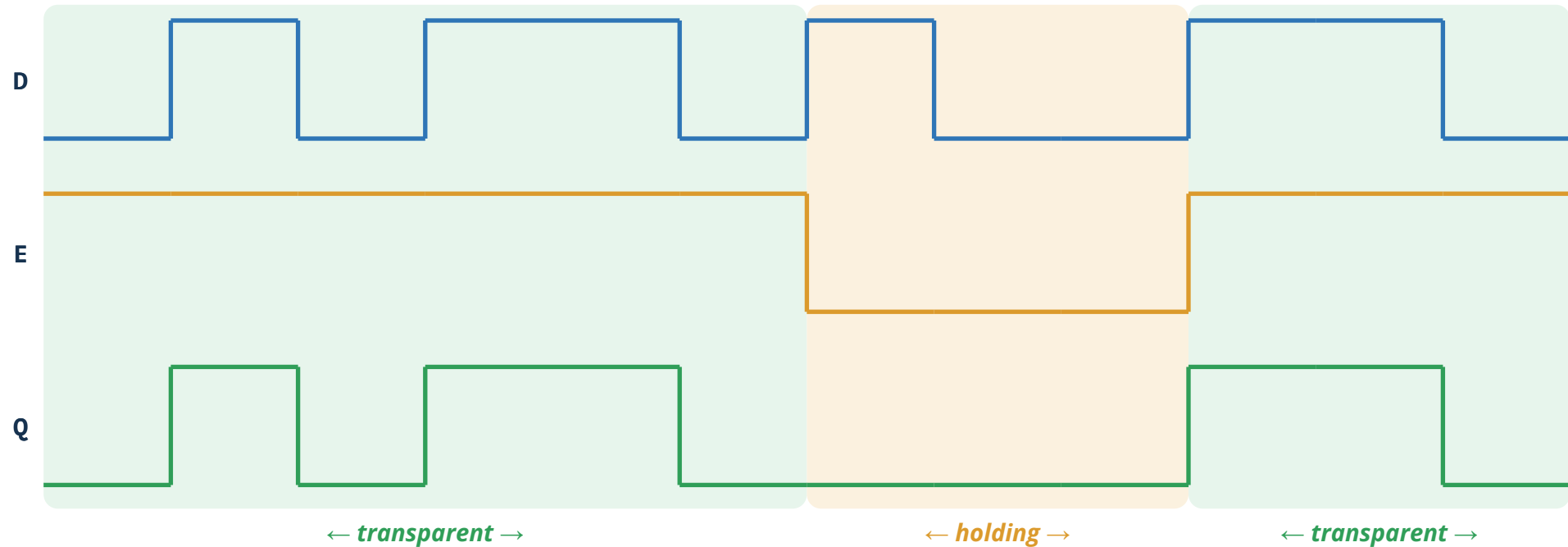
$S=0, R=1$. SR latch is RESET. $Q = 0$.

While E is high, Q follows D — we call this 'transparent'. While E is low, Q holds.



D Latch · Transparency

While E is high, Q tracks D step-for-step. While E is low, whatever D was at the falling edge of E is what Q holds.



During the green windows, Q is a copy of D. During the amber window, Q remembers the last D from before E fell.

D Latch · Verilog

Two assigns to compute S_{int} and R_{int} from D and E . Then reuse the SR latch as a submodule.

d_latch.v

// Transparent D latch built from an SR latch

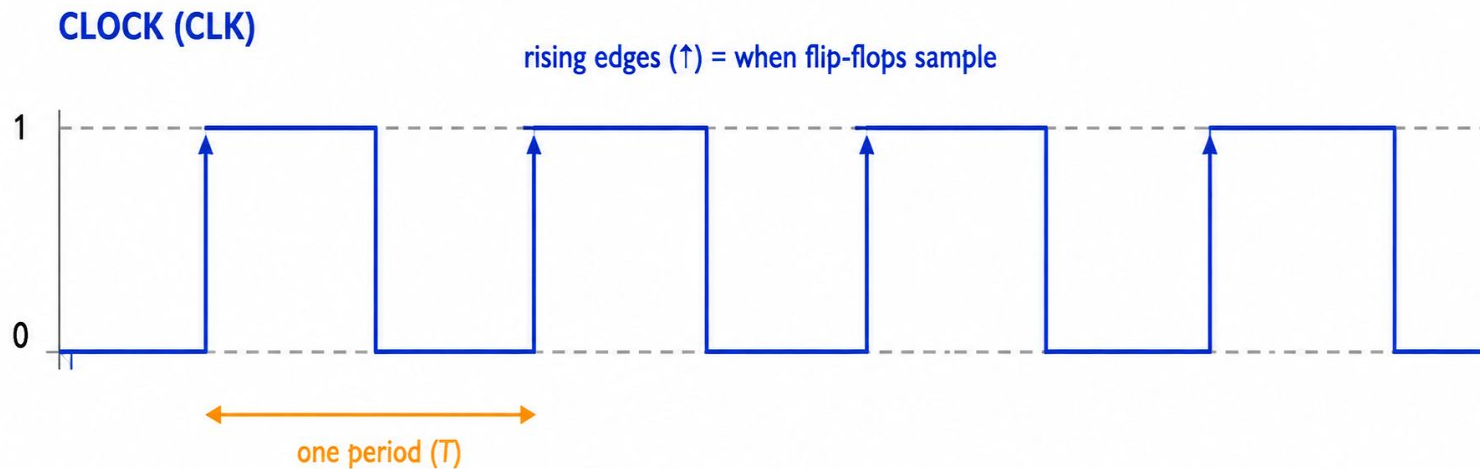
```
module d_latch (  
    input  wire d,  
    input  wire e,  
    output wire q,  
    output wire qbar  
);  
    wire s_int, r_int;  
    assign s_int = d & e;  
    assign r_int = ~d & e;  
    sr_latch core(.s(s_int), .r(r_int),  
                  .q(q), .qbar(qbar));  
endmodule
```

NOTES

- S and R are mutually exclusive by construction.
- When $E=0$, both s_{int} and r_{int} are 0 — the SR latch holds.
- Reusing `sr_latch` keeps the design hierarchical — like the FA from 2 HAs in Lab 3.
- Compile with both .v files: `iverilog -o d.vvp sr_latch.v d_latch.v d_tb.v`

The Clock: The System Heartbeat

One periodic signal synchronises every flip-flop in the chip.



Frequency: $f = 1 / T$

Example: A 3 GHz CPU has $T = 0.33$ nanoseconds $\rightarrow$ 3 billion edges per second.

WHY ONE CLOCK

Every flip-flop updates on the same edge.

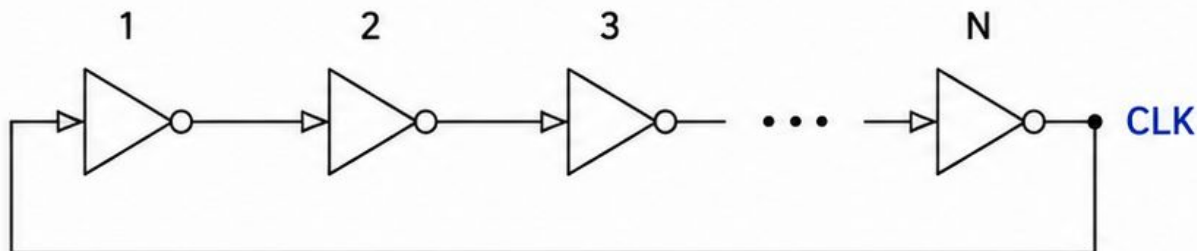
That shared instant is what keeps a whole chip's state consistent — everyone steps together.

The clock is the drummer the entire orchestra follows. Lose sync, lose everything.

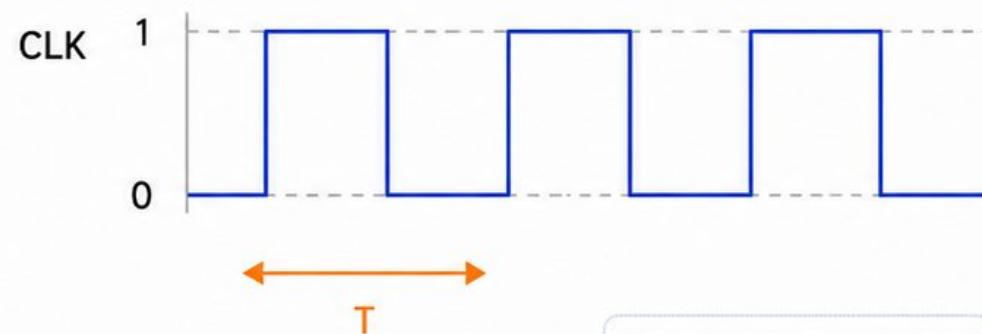
How a Clock is Made Using Logic Gates

A clock is generated using an **oscillator** – a feedback loop of inverters (NOT gates).

1. Ring Oscillator using Inverters (Odd number)



2. Output Clock Waveform



Frequency: $f = 1 / T$

3. How it Works

- Signals pass through the inverters and are inverted.
- The feedback loop with an odd number of inverters causes continuous toggling.
- The output of any inverter is a continuous square wave (clock).

Key Points

- Use an odd number of inverters (3, 5, 7, ...).
- Each inverter adds a small delay.
- Total delay around the loop decides the clock period (T).

Level-Triggered vs Edge-Triggered

The difference between 'listen the whole time' and 'listen for one instant'.

LEVEL-TRIGGERED (latch)

- Active during an entire clock LEVEL (high or low).
- Transparent — output tracks input the whole window.
- Simpler, fewer gates.
- Risky: data can slip through unintentionally.

EDGE-TRIGGERED (flip-flop)

- Active only at the clock EDGE (rising or falling).
- Samples input at one instant, then locks.
- More gates, but predictable.
- The safe choice for pipelines & registers.

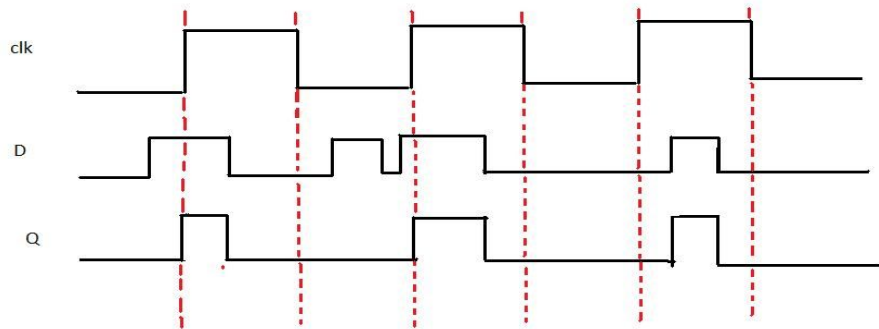
↑ rising edge

An edge is a MOMENT (0→1 transition), not a duration. That's why edge-triggering samples cleanly.

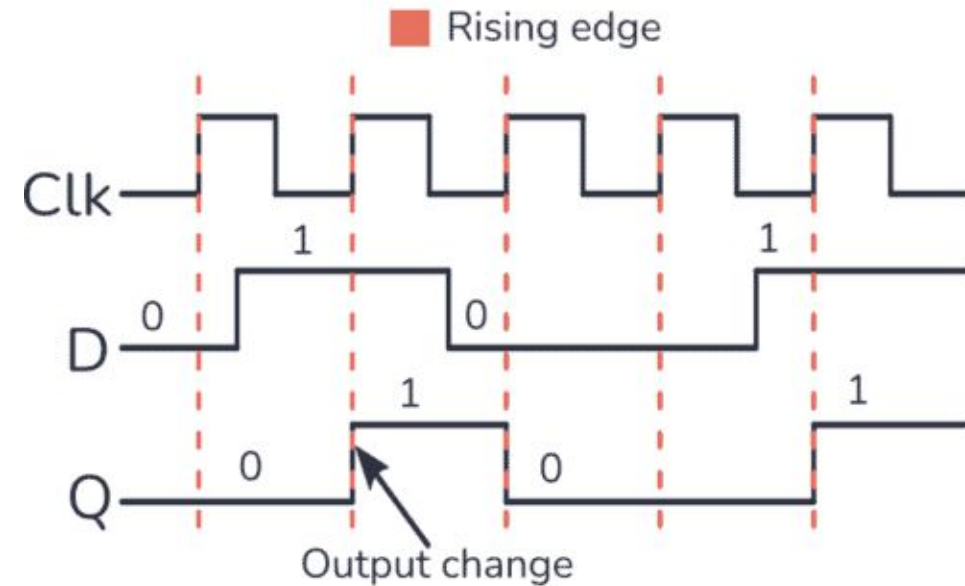
Transparency Is Sometimes a Problem

Anything that wiggles on D while E is high lands on Q. Glitches included. That's why CPUs don't use latches for the critical path.

D LATCH · TRANSPARENT



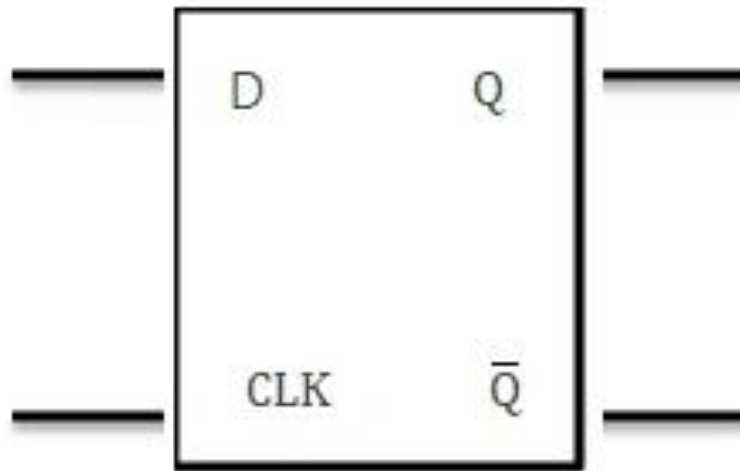
D FLIP-FLOP · EDGE TRIGGERED



Same noisy D. Same clock. Latch propagates everything. Flip-flop ignores everything except the value at the rising edge.

The D Flip-Flop

Samples D on the rising clock edge, then holds it for the whole cycle.



The small triangle on the clock input is the universal symbol for 'edge-triggered'.

THE RULE

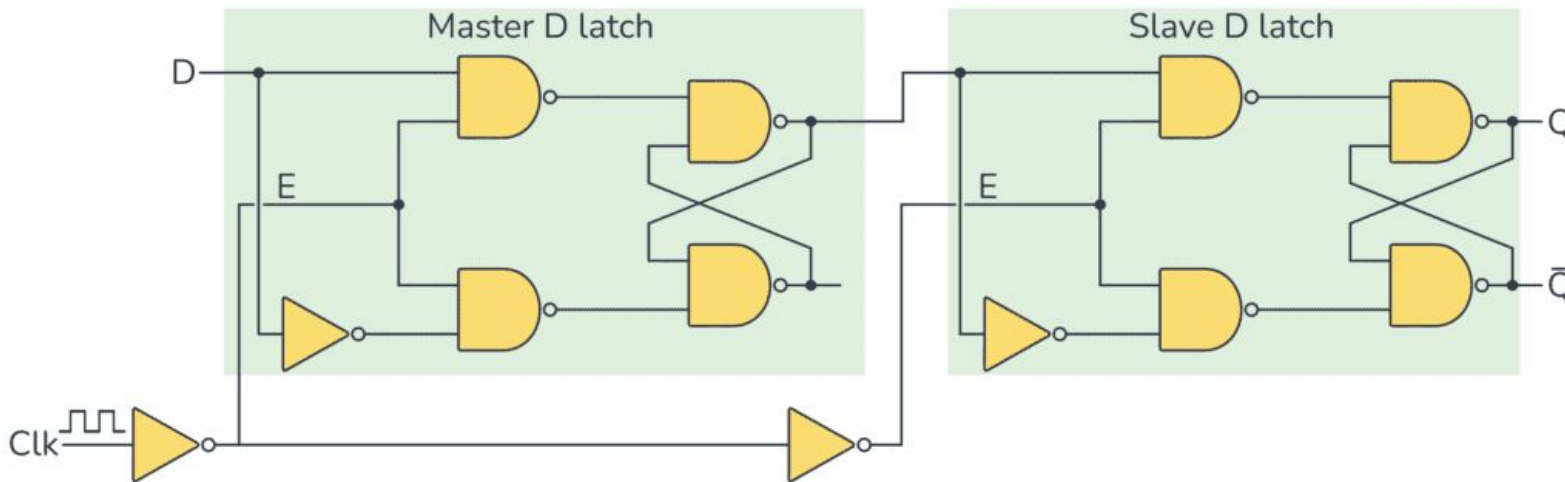
At each rising edge of CLK:
 $Q \leftarrow D$ (sampled once).

Between edges: Q is frozen, no matter what D does.

So D can wiggle all it wants — only its value AT the edge is captured. This solves the transparency problem completely.

How Edge-Triggering Works: Master-Slave

Two latches in series on opposite clock phases — data can only advance one step per cycle.



THE HAND-OFF

The two latches are **NEVER** transparent at the same time.

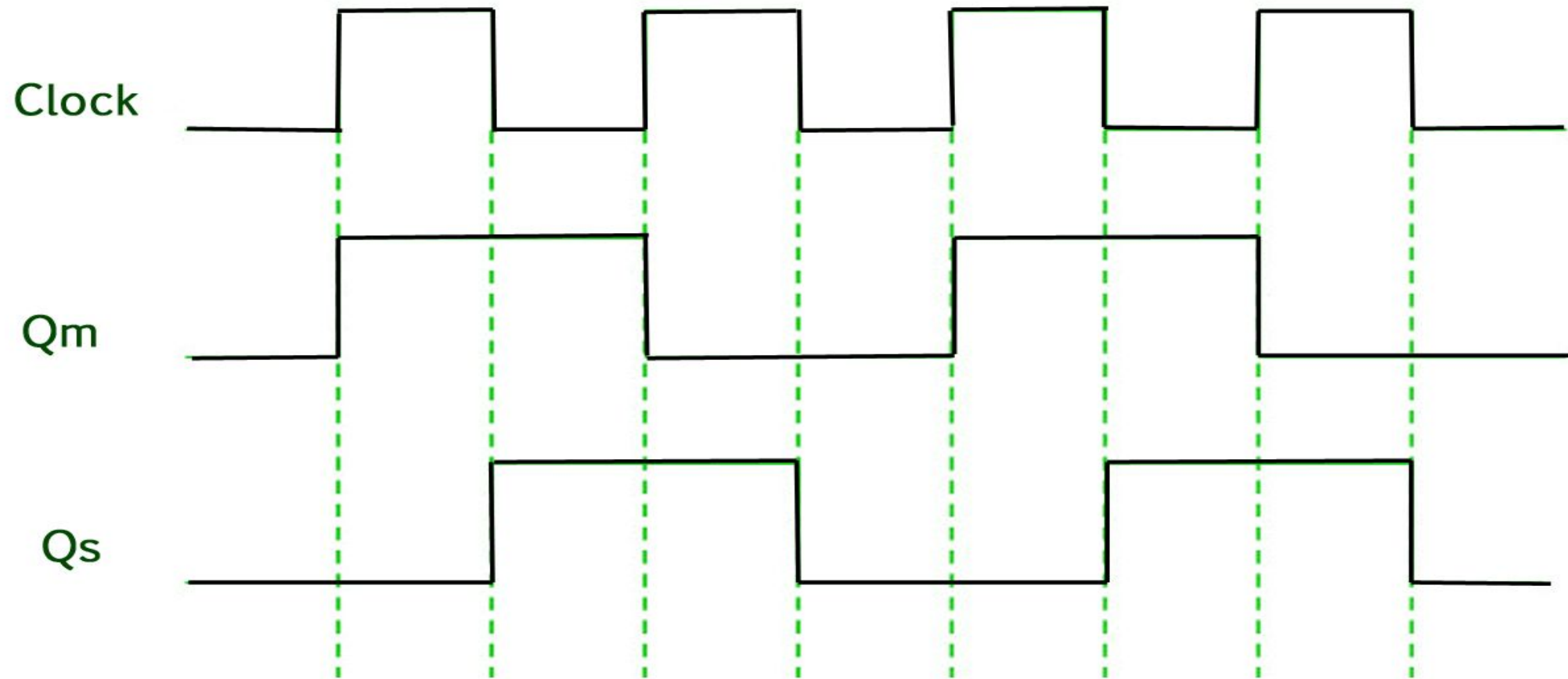
When the master listens, the slave is locked — and vice versa.

So data can't race straight through. It's a two-stage airlock for one bit.

Master captures D while CLK is high; slave passes it on the falling edge. Result: one clean update per cycle.

From Latch to Flip-Flop · The Clock

An edge-triggered flip-flop is a latch that opens for an instant on the clock edge — too briefly for glitches to pass.



D Flip-Flop · Verilog

Procedural this time. The `always @(posedge clk)` block tells the simulator: 'evaluate this only at the rising edge of clk'.

```
d_ff.v
// Positive-edge-triggered D flip-flop
module d_ff (
    input wire clk,
    input wire d,
    output reg q // reg, not wire
);
    always @(posedge clk) begin
        q <= d; // nonblocking
    end
endmodule
```

THREE THINGS NEW

- output reg — q is now a procedural variable. wire can't be assigned inside always.
- always @(posedge clk) — the block runs once each time clk goes 0→1. Never any other time.
- <= (nonblocking) — the update happens at the end of the time step. Next slide explains why this matters.

D Flip-Flop · Testbench

Drive a clock. Drive D with deliberate noise between edges. Verify Q only changes on rising edges.

ff_tb.v

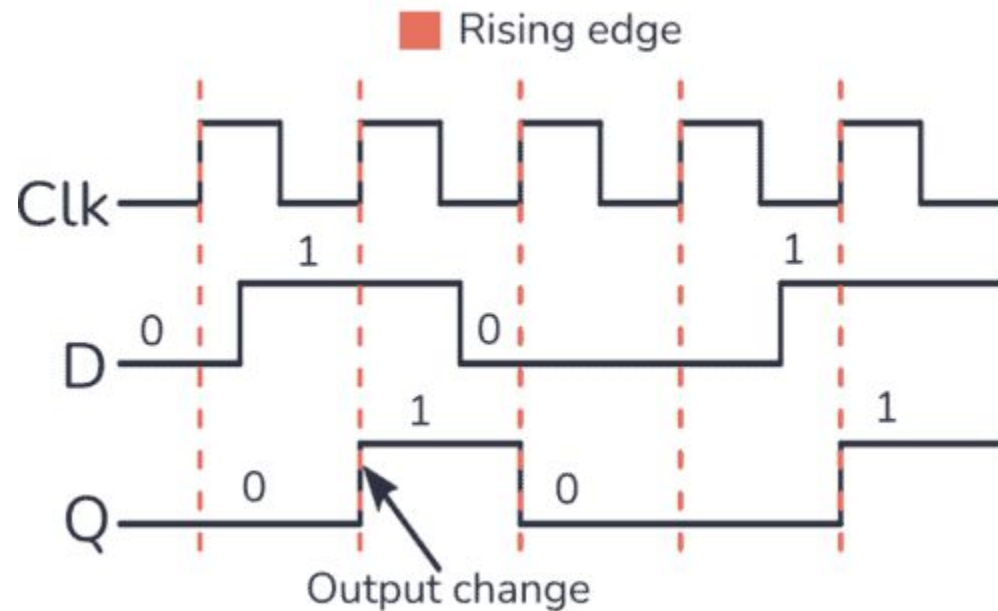
```
module ff_tb;
  reg clk, d; wire q;
  d_ff dut(.clk(clk), .d(d), .q(q));
  initial clk = 0;
  always #5 clk = ~clk; // period = 10 ns
  initial begin
    $dumpfile("ff.vcd"); $dumpvars(0, ff_tb);
    d = 0; #3; // before 1st edge
    d = 1; #1; // glitch up
    d = 0; #1; // glitch down - edge at t=5
    d = 1; #6; // settle to 1
    d = 1; #10; // hold across 2 edges
    d = 0; #10;
    $finish;
  end
endmodule
```

EXPECTED Q

- Q stays 0 until t=5.
- At t=5 (rising edge), D was 0 → Q stays 0.
- At t=15, D = 1 → Q goes to 1.
- All intermediate glitches on D are ignored.

D Flip-Flop · What the Waveform Shows

CLK ticks. D wiggles. Q updates only at the up-arrows. Everything else gets thrown away.



Notice Q ignores all the glitches on D between edges. It samples only at the four red arrows.

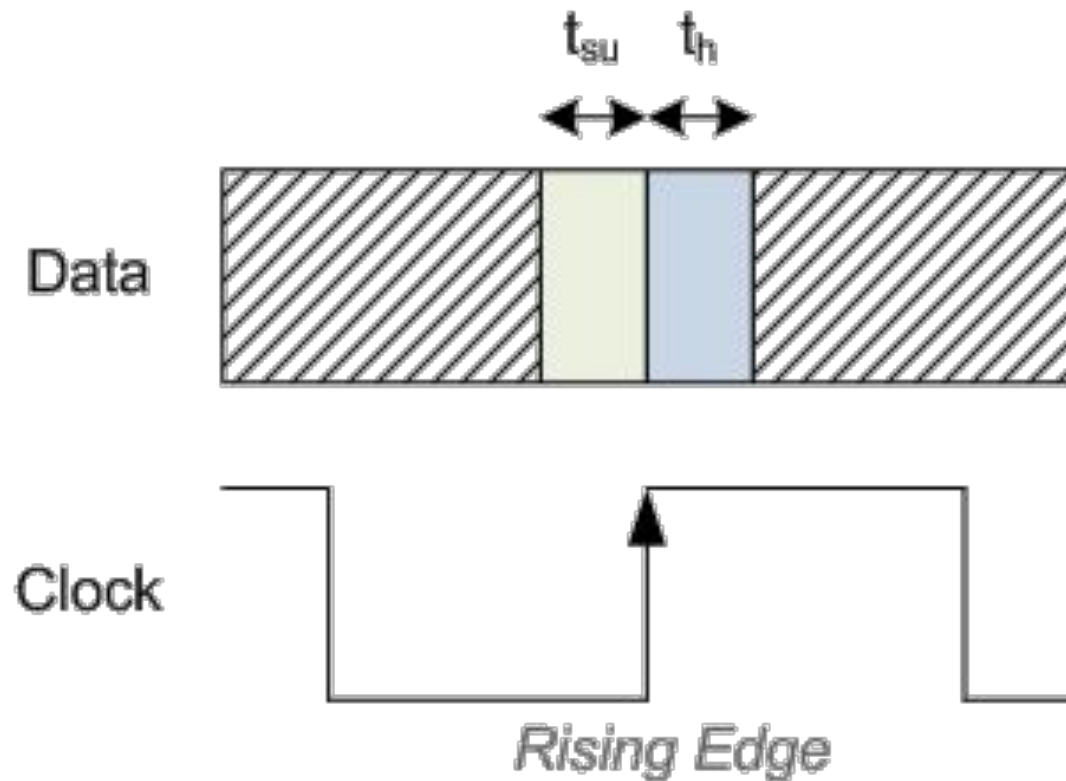
Side by Side · D Latch vs D Flip-Flop

Same circuit symbol, very different behaviour. Memorize this table — it's the most-asked digital-design interview question.

	D Latch	D Flip-Flop
Control signal	Enable (level)	Clock (edge)
Output during 'active'	Q follows D continuously	Q samples once, then holds
Transparent?	Yes — D wiggles, Q wiggles	No — Q is opaque between edges
Glitches on D propagate?	Yes	No
Verilog body	assign (or always @(*))	always @(posedge clk)
Output type	wire	reg
Used in modern CPUs?	Rarely — special cases only	Everywhere — pipelines, registers

Setup Time & Hold Time

Data must be STABLE in a window around the clock edge — before and after.



THE ANALOGY

Taking a photo of a moving object.

The subject must hold still just before AND just after the shutter clicks, or the picture blurs.

Same for a flip-flop: change D inside the setup/hold window and the captured bit is a blur — undefined.

Setup & Hold Time · The Fine Print

The flip-flop only samples reliably if D is stable for a small window around the rising edge. Violate that window and Q goes metastable.

t_{setup}

how long D must be
stable BEFORE the edge

t_{hold}

how long D must be
stable AFTER the edge

t_{cq}

delay from the edge
to Q being valid

WHAT GOES WRONG

If D changes inside the $[\text{edge} - t_{\text{setup}}, \text{edge} + t_{\text{hold}}]$ window, the flip-flop captures neither 0 nor 1 cleanly. Q enters a metastable state — somewhere between rails — and may stay there for many gate-delays before snapping to one or the other. This is why CPU pipelines have synchronizers at every clock-domain crossing.

When to Use a Latch

Flip-flops won the war, but latches still show up in three specific places. Knowing why matters more than memorizing them.

01

SRAM cells

Every bit of cache is two cross-coupled inverters — essentially an SR latch. There are billions of these on a modern die.

02

Time-borrowing

In pipelines too tight for one clock period, designers insert a latch between two flop stages. The latch lets a slow operation 'borrow' time from the next cycle.

03

Asynchronous resets

Reset logic is often latched, not flopped — you want to be sure the reset signal sticks even if the clock isn't running yet.

Everywhere else — pipeline registers, control state, the PC, ALU result registers — it's flip-flops.

Common Errors · And the Fix

Five traps that catch most students. Glance now and save yourself half an hour of waveform-staring.

- | | |
|--|--|
|  Declared q as wire inside the flip-flop | Procedural assignments need reg. Verilog will refuse to compile and the error message points at the wrong line. Change wire to reg. |
|  Used `=` instead of `<=` in posedge block | Compiles fine, often even simulates fine. But cascaded stages get the wrong values when clk-skew matters. Use <= in every posedge block, every time. |
|  Forgot the begin/end on multiline blocks | Verilog only ties the first statement to the @(posedge). Subsequent lines run as combinational logic. Wrap multi-line bodies in begin ... end. |
|  Drove S and R high simultaneously in SR test | iverilog will report 'X' on q after release. That's not a bug — it's the forbidden state. Re-read slide 9 and gate your inputs. |
|  Built a flip-flop and called it a latch | If your D propagates only at posedge clk, you have a flip-flop. A real latch uses always @(*) or continuous assigns with an enable wire. |

Why This Matters · What's Next

Every register on the CPU you'll build later is a row of D flip-flops. Every pipeline stage is a flop. Today is the foundation of all of that.

TODAY

One bit, one flop

You built the elementary unit of digital memory. From here, everything scales.

TOMORROW

Registers, banks

Stack 32 flip-flops side by side, share a clock — you have a 32-bit register. Build 32 of those, you have the register file.

REAL CHIPS

Pipelines and pacing

Every stage boundary in a CPU pipeline is a row of flops. Their setup/hold determines the max clock frequency.

BEYOND

FSMs and control

Combine flops with combinational logic and you have a finite-state machine. That's how every controller in a CPU works.

You just learned, in 90 minutes, the storage cell that ships in every CPU on Earth.

Rapid-Fire Recap

Three questions — shout the answers.

Q1**S = R = 1 on an SR latch is called...****forbidden****Q2****A D latch is transparent when E equals...****1****Q3****A D flip-flop samples D on the clock's...****rising edge**

All three instant? You've got the foundation — Lab 06 builds registers and register files from exactly this cell.



One latch. One flip-flop. One bit that remembers.

Next: registers, register files, and the first real piece of a datapath.



**Thanks
for
watching!**